

WEA

Widget meteo con effetti vetro dell'800

Documentazione tecnica — Android AppWidget nativo — Java

Android API 21+

Java 8

Canvas / Bitmap

OpenWeatherMap API

Branch: `claude/android-glass-widget-VKGIS`

Indice

1. Concetto e obiettivo
2. Architettura generale
3. Pipeline di rendering
4. Generazione sfondo — Cielo e mare
5. Effetti nella scena (dietro il vetro)
6. Effetto vetro dell'800
7. Effetti sul vetro (pioggia, tinta)
8. Ora e data
9. Cornice in legno
10. Sistema di animazione — RenderService
11. Meteo — WeatherManager e WeatherUpdateService
12. Configurazione e test effetti
13. File sorgente

1. Concetto e obiettivo

WEA è un **widget per la home screen Android** che simula la vista attraverso un vetro imperfetto dell'Ottocento su un paesaggio marino mediterraneo. Ora e data sono sempre leggibili e nitide; lo sfondo e gli effetti cambiano in tempo reale in base all'ora del giorno e alle condizioni meteo.

Caratteristica	Dettaglio
Tipo	AppWidgetProvider Android (home screen widget)
Rendering	Bitmap generata via Canvas Java, aggiornata via RemoteViews
Aggiornamento	Ogni 2 secondi (RenderService foreground)
Meteo	OpenWeatherMap API 2.5, posizione GPS o città
Dimensioni	2 righe fisse, larghezza ridimensionabile (resizeMode=horizontal)
API minima	Android 5.0 (API 21)

2. Architettura generale

Componente	Tipo Android	Responsabilità
SeaWidget	AppWidgetProvider	Coordinatore: riceve tick, aggiorna widget, gestisce AlarmManager
WidgetRenderer	Classe statica	Genera l'intera Bitmap (sfondo + effetti + vetro + testo + cornice)
RenderService	Foreground Service	Tick ogni 2 s → chiama updateAllWidgets() per animazione fluida
WeatherUpdateService	IntentService	Fetch HTTP OpenWeatherMap, salva condizione in SharedPreferences
WeatherManager	Classe statica	Legge/scrive condizione meteo; gestisce override di test
ConfigActivity	Activity	Imposta città/GPS/API key; pannello test effetti
BootReceiver	BroadcastReceiver	Riavvia RenderService e AlarmManager dopo il reboot

Il widget non usa Gradle, AGP, o librerie esterne. La build è uno script bash che chiama direttamente `aapt` , `javac` , `dx` , `zipalign` e `apksigner` .

3. Pipeline di rendering

Ogni 2 secondi `WidgetRenderer.render()` produce una nuova Bitmap che viene passata a `RemoteViews.setImageViewBitmap()`. La pipeline è strettamente sequenziale e produce l'effetto di "guardare attraverso il vetro":

```
render(ctx, dpWidth, dpHeight)
|
|─ generateSeaSky() → Bitmap base: cielo + glow + costa + foschia + mare
|
|─ addSceneEffects()
→ disegna SULLA bitmap base (prima della distorsione)
|   |─ drawRainInScene() pioggia nella scena, 2 layer
|   |─ drawRainRipples() cerchi d'impatto sull'acqua
|   |─ drawWaterSparkles() riflessi del sole (CLEAR, giorno)
|   |─ drawBirds() gabbiani in volo (CLEAR, giorno)
|   |─ drawLeaves() foglie portate dal vento
|   |─ drawStars() stelle con brillucciatura (notte/dusk)
|
|─ applyGlassDistortion() → nuova Bitmap: distorsione pixel-per-pixel
|
|─ drawGlassSurface() → tinta minerale, vignette, bolle d'aria, striature
|
|─ drawRainOnGlass()
→ pioggia sul vetro (3 layer + wet tint) [solo RAINY/STORMY]
|
|─ drawTimeDate() → ora e data SEMPRE nitidi (sopra tutto)
|
|─ drawWindowFrame() → cornice in legno invecchiato
```

Il risultato è una singola Bitmap che viene inviata al launcher tramite RemoteViews. Ogni effetto usa `System.currentTimeMillis()` come sorgente di animazione, garantendo continuità anche dopo il riavvio del servizio.

4. Generazione sfondo — Cielo e mare

4.1 Ore del giorno

La funzione `getTimeOfDay()` classifica l'ora in 7 fasce:

Fascia	Ore	Palette cielo
DAWN	5–7	Viola scuro → rosso-bordeaux → arancio caldo
MORNING	7–11	Blu notte → blu vivo → celeste chiaro
NOON	11–14	Blu cobalto → azzurro → celeste brillante
AFTERNOON	14–17	Blu medio → oro caldo all'orizzonte
SUNSET	17–20	Viola → rosso bruciato → arancio intenso
DUSK	20–22	Quasi nero → blu notte profondo
NIGHT	22–5	Nero assoluto → blu notte scurissimo

4.2 Cielo — `drawSky()`

Tre stop di gradiente lineare verticale (top → mid → orizzonte). Il meteo CLOUDY/WINDY desatura tutti i colori del 45% verso grigio; RAINY/STORMY li scurisce del 65–70% verso grigio-blu scuro.

4.3 Glow sole/luna — `drawSunMoonGlow()`

Un `RadialGradient` molto esteso simula il bagliore diffuso senza mai disegnare un disco. La posizione del sole cambia per fascia oraria (basso a sinistra all'alba, alto al centro a mezzogiorno, basso a destra al tramonto). Di notte/crepuscolo appare un glow lunare argentato in alto a destra. Con STORMY il glow è soppresso.

4.4 Silhouette della costa — `drawCoastSilhouette()`

Una path con due curve di Bézier cubiche disegna profili collinari lontani appena sopra la linea dell'orizzonte (a `horizonY = h * 0.52`). Il colore è semi-trasparente e varia con l'ora; in caso di pioggia/temporale la visibilità è ridotta al 40%.

4.5 Foschia orizzonte — `drawHorizonHaze()`

Un gradiente lineare simmetrico (Mirror) attorno alla linea di orizzonte simula la diffusione atmosferica. Il colore cambia con l'ora: arancio caldo al tramonto, blu-bianco a mezzogiorno, viola a notte.

4.6 Mare — drawSea()

Gradiente verticale dal colore di superficie al colore abissale. Sopra di esso:

- **drawWaveLines()**: 14 linee ondulate con prospettiva (scalate in larghezza e opacità in base alla distanza dall'orizzonte).
- **drawSeaReflection()**: colonna trapezoidale semi-trasparente che simula il riflesso del sole/luna sull'acqua.

5. Effetti nella scena (dietro il vetro)

Questi effetti sono disegnati sulla bitmap base *prima* della distorsione del vetro: appaiono quindi leggermente deformati dal vetro stesso, aumentando la coerenza visiva.

5.1 Pioggia nella scena — drawRainInScene()

Implementa i principi *multi-layer* e *velocity-aligned streaks*:

Layer	Posizione	Caratteristiche
Lontano (cielo)	Sopra l'orizzonte	Alpha 10, spessore 0.4, corte, quasi verticali
Vicino (mare)	Sotto l'orizzonte	Alpha 16, spessore 0.55, scalate in prospettiva, angolate dal vento

L'angolo varia: 7° per RAINY, 18° per STORMY. Il seed temporale cambia ogni 150 ms, rendendo la pioggia visibilmente mobile.

5.2 Cerchi d'impatto — drawRainRipples()

Ellissi che si espandono nel tempo (`now % 600ms`) con alpha inversamente proporzionale al raggio. La prospettiva è simulata: i cerchi vicini all'orizzonte sono più piccoli ed ellittici; quelli in primo piano più grandi e quasi circolari.

5.3 Riflessi del sole — drawWaterSparkles()

14–26 riflessi a forma di croce (+x) sul mare. Ogni riflesso ha un proprio periodo di pulsazione (2.5–7.5 s) e una fase casuale, quindi lampeggiano in modo indipendente. Solo con CLEAR di giorno.

5.4 Gabbiani in volo — drawBirds()

7 gabbiani con parametri fissi (hardcoded) per garantire animazione deterministica:

```
float[][] BIRDS = {
    {cicloMs, frazioneY_rispetto_orizzonte, sfasamento_0..1, scalaDimensione},
    {28000f, 0.65f, 0.00f, 1.00f}, // gabbiano 1: ciclo 28 s
    {35000f, 0.73f, 0.35f, 1.35f}, // gabbiano 2: ciclo 35 s
    ...
};
```


La posizione X è `progress = (now + phaseOffset) % cycle / cycle` → il gabbiano scorre da sinistra a destra e ricomincia. Tutti le ali battono in sincronia (`flapDip = |sin(now % 1600ms * PI)|`), simulando un banco.

5.5 Foglie — drawLeaves()

Le foglie si muovono con velocità e rotazione proporzionali all'intensità del vento:

Condizione	Numero foglie	Velocità px/s
STORMY	12–20	95
WINDY	7–12	58
RAINY	3–7	30
CLEAR	1–4	16

Ogni foglia ha posizione di partenza casuale (seed orario), velocità (x,y) e velocità di rotazione individuali. La posizione attuale è `x0 + vx * (now % cycle / 1000)` . Le foglie sono scalate in base alla posizione verticale: più grandi in basso (vicino allo spettatore), più piccole in alto (lontane).

5.6 Stelle — drawStars()

30–50 stelle in posizioni fisse (seed orario). Il 28% "brilluccica" con un periodo individuale (2–5.5 s) e una fase casuale: `bright = base * (0.3 + 0.7 * |sin(now/period * PI + phase)|)` . Il 12% sono stelle "grandi" con un bagliore a croce.

6. Effetto vetro dell'800

6.1 Distorsione delle onde — `applyGlassDistortion()`

Operazione pixel-per-pixel: ogni pixel della bitmap risultante viene campionato da una posizione spostata nella bitmap sorgente. Lo spostamento è calcolato da una somma di onde sinusoidali a frequenze diverse, simulando le imperfezioni di un vetro soffiato a mano:

```
double dx = 3.2 * sin(y * 0.014 + 0.53) + 1.8 * cos(x * 0.011 + y * 0.007);
double dy = 2.8 * cos(x * 0.013 + 1.21) + 1.6 * sin(y * 0.010 + x * 0.006);
// Micro-turbolenza ad alta frequenza:
double dx2 = 0.9 * sin(x * 0.048 + y * 0.031);
double dy2 = 0.7 * cos(y * 0.042 + x * 0.022);
```

La funzione usa `getPixels()` / `setPixels()` in blocco per minimizzare le chiamate JNI. Per un widget tipico di 360×220 px: ~80.000 pixel, ~320.000 operazioni trigonometriche — eseguito in <50 ms su hardware moderno.

6.2 Superficie del vetro — `drawGlassSurface()`

Quattro livelli sovrapposti dopo la distorsione:

1. **Tinta minerale:** rettangolo verde-grigio semi-trasparente (`argb(20, 25, 70, 15)`) su tutta la superficie.
2. **Vignette:** `RadialGradient` dal centro verso i bordi, nero con alpha 70.
3. **Riflesso superiore:** gradiente lineare bianco → trasparente nell'10% superiore.
4. **Bolle d'aria** (10 posizioni fisse): cerchi con bordo verde-grigio, riflesso interno bianco — simulano le bolle intrappolate nel vetro durante la fabbricazione.
5. **Striature di colata** (3 path): linee cubiche verticali sottilissime, bianco con alpha 10–18 — simulano le striature da colata del vetro soffiato.

7. Effetti sul vetro (pioggia, tinta)

Disegnati *dopo* la distorsione: appaiono nitidi, come se fossero fisicamente depositati sul vetro.

7.1 Pioggia sul vetro — `drawRainOnGlass()`

Tre layer sovrapposti più una tinta wet:

Layer	Elementi	Effetto
Tinta wet	Rettangolo pieno	Overlay blu-grigio <code>argb(20, 85, 120, 160)</code> su tutto

Lontano	40 tratti	Alpha 28, spessore 0.5 px, brevi (10–48 px scalati)
Medio	22 tratti	Alpha 44, spessore 0.9 px, più lunghi (28–88 px)
Vicino	14 gocce	Lacrime con highlight bianco + streak che cola sotto

L'angolo è fisso a 7° (vento sul vetro). Il seed cambia con `now` direttamente → la pioggia si muove visibilmente ogni 2 s.

8. Ora e data

Disegnati per ultimi, sempre in primo piano, *mai* distorti dal vetro:

- **Ora**: font sans-serif bold, dimensione 30% dell'altezza del widget, ombra morbida, centrata.
- **Data**: font normal, 13% dell'altezza, spaziatura lettere larga, centrata sotto l'ora.
- Formato data in italiano: `Lun 5 Gen` (giorno abbreviato + numero + mese abbreviato).

9. Cornice in legno — `drawWindowFrame()`

Quattro barre di spessore `t = min(w,h) * 0.085` con gradiente lineare che simula il legno invecchiato (marrone chiaro → scuro). Dettagli:

- **Grana del legno**: linee sottili orizzontali e verticali ogni 5.5 px, bianco con alpha 22.
- **Effetto 3D**: bordo interno superiore/sinistro chiaro, inferiore/destro scuro.
- **Ombra proiettata**: gradiente scuro verso l'interno del vetro.
- **Blocchi agli angoli**: quadrati più scuri.
- **Chiodini**: 4 cerchi nei pressi degli angoli, dorati semi-trasparenti.

10. Sistema di animazione — RenderService

10.1 Perché un Service

Un AppWidget è una Bitmap statica: senza un processo attivo che la rigeneri periodicamente, non ci può essere animazione. Android mette a disposizione `AlarmManager` (minimo ~1 minuto con `setInexactRepeating`) o un **foreground Service** con un `Handler` interno per aggiornamenti più frequenti.

10.2 RenderService

```
handler.postDelayed(tick, 2000L); // ogni 2 secondi

Runnable tick = () -> {
    SeaWidget.updateAllWidgets(context); // → WidgetRenderer.render()
    handler.postDelayed(this, 2000L);
};
```

Poiché è un foreground Service, deve mostrare una notifica persistente (obbligatoria da Android 8, API 26). La notifica è a priorità minima (`IMPORTANCE_MIN`) e praticamente invisibile.

10.3 Robustezza

- Il servizio usa `START_STICKY`: Android lo riavvia se viene terminato.
- Il tick del meteo (`ACTION_WEATHER_TICK`, ogni minuto) chiama anche `RenderService.start()` come fallback: se il servizio è stato killato da battery optimization, riparte entro 60 s.
- Il `BootReceiver` riavvia il servizio dopo ogni reboot.

10.4 Seed temporali

Variabile	Formula	Cambia ogni	Usata per
<code>now</code>	<code>currentTimeMillis()</code>	Ogni render (2 s)	Posizione uccelli, pioggia, stelle, foglie
<code>timeSeed</code>	<code>now / 60000</code>	1 minuto	Sfondo (onde, fase glow)
<code>hourSeed</code>	<code>now / 3600000</code>	1 ora	Posizioni fisse sparkles/stelle/foglie

11. Meteo — WeatherManager e WeatherUpdateService

11.1 Condizioni meteo

Enum	Codici OWM	Effetti abilitati
CLEAR	800	Sole, uccelli, sparkles, foglie (poche)
CLOUDY	801–804, 600–799	Cielo desaturato, foglie medie
RAINY	300–599	Pioggia 3-layer, ripples, vetro bagnato, foglie
STORMY	200–299	Come RAINY, pioggia intensa, cielo scuro, no glow
WINDY	CLEAR/CLOUDY + vento > 8 m/s	Cielo parziale, foglie abbondanti

11.2 Flusso di aggiornamento meteo

```
AlarmManager (ogni 60 s)
→ SeaWidget.onReceive(ACTION_WEATHER_TICK)
→ WeatherUpdateService.onHandleIntent()
→ HTTP GET api.openweathermap.org/data/2.5/weather
→ parse JSON (weather.id, wind.speed)
→ WeatherManager.saveCondition() → SharedPreferences
→ SeaWidget.updateAllWidgets()
```

11.3 Test override

Un intero `weather_test_override` nelle SharedPreferences (–1 = nessuno, 0–4 = condizione) permette di forzare la condizione senza aspettare il meteo reale.

`WeatherManager.getCondition()` legge prima questo valore.

12. Configurazione e test effetti

La `ConfigActivity` si apre toccando il widget. Permette di:

- Inserire manualmente la città (fetch meteo per nome)

- Rilevare e salvare la posizione GPS (fetch per lat/lon)
- Modificare la chiave API OpenWeatherMap
- Forzare istantaneamente una condizione meteo per testare gli effetti

I pulsanti di test (Sereni / Nuvoloso / Ventoso / Pioggia / Temporale / Meteo reale) aggiornano l'override e chiamano immediatamente `updateAllWidgets()`: il widget cambia aspetto in meno di 2 secondi.

13. File sorgente

File	Righe	Funzione
<code>WidgetRenderer.java</code>	~850	Intera pipeline grafica
<code>SeaWidget.java</code>	~120	AppWidgetProvider, scheduling
<code>RenderService.java</code>	~100	Foreground service, tick 2 s
<code>WeatherUpdateService.java</code>	~90	Fetch HTTP meteo
<code>WeatherManager.java</code>	~60	Gestione condizione + override test
<code>ConfigActivity.java</code>	~140	UI configurazione e test
<code>BootReceiver.java</code>	~15	Restart su boot
<code>build.sh</code>	~80	Build manuale (aapt → javac → dx → zip → sign)
<code>AndroidManifest.xml</code>	~50	Dichiarazione componenti e permessi
<code>activity_config.xml</code>	~190	Layout configurazione
<code>widget_layout.xml</code>	~10	Layout widget (solo ImageView)

Dipendenze esterne: nessuna. Tutto il rendering usa unicamente le API del framework Android standard (android.graphics.Canvas, Bitmap, Paint, Path, LinearGradient, RadialGradient). Non è usato nessun engine grafico, libreria di animazione o framework di terze parti.